Non-Negative Conjugate Gradients

Thomas Schmelzer[1] and Martin Stoll[2]

[1]Jebel Quant Research, Abu Dhabi
[2]Faculty of Mathematics, TU Chemnitz

July 2, 2026

Abstract

The conjugate gradient method (CG) solves a symmetric positive definite (SPD) linear system $Ax = b$; it does not, on its own, respect the inequality $x \geq 0$. This note develops *non-negative conjugate gradients*: a solver for the bound-constrained quadratic

$$\min_{x \geq 0} \ \tfrac{1}{2}\, x^\top A x - b^\top x, \qquad A \succ 0,$$

and its equality-augmented (a general linear system $Bx = c$) and least-squares variants, that enforces $x \geq 0$ by wrapping CG in an outer active-set loop while retaining CG's $O(\sqrt{\kappa})$ Krylov convergence, where $\kappa = \kappa(A)$ is the spectral condition number.

The wrapper is a primal-dual active-set method. It solves the unconstrained system over a working set of *free* variables, releases any that come back negative (the primal step), re-admits any bound variable whose reduced gradient is negative (the dual step), and restarts CG on the modified free set each time. These toggles are exactly the principal pivots of the linear complementarity problem $(A, -b)$; because A is a P-matrix, every principal submatrix is invertible and each pivot is well defined. The central result concerns termination: guarding a fast block-pivot path with a least-index Bland fallback, we prove the loop reaches the unique global minimiser in finitely many outer steps with *no* non-degeneracy assumption (Theorem 5.1) – removing the hypothesis that classical block-principal-pivoting termination arguments require, while a synthetic study shows the fallback is provably necessary yet never entered on generic data. The guarantee is robust to inexactness: once CG's residual tolerance is tied to the decision threshold, the inexact loop provably visits the same free sets as the exact one (Lemma 5.1). A loop-free projected-gradient comparator, which enforces the constraints simultaneously but forgoes the Krylov subspace and so converges at the slower $O(\kappa)$ rate, is analysed alongside it.

Each inner solve is matrix-free. When $A = M^\top M$ arises as a Gram matrix, the operator $v \mapsto M^\top(Mv)$ is applied without ever forming A, so CG on the normal equations costs two products with M per iteration and no $O(n^2)$ storage. Because the inner rate is governed by κ, any regularisation or preconditioner that compresses the spectrum – including the ridge/Tikhonov splitting $A \mapsto (1-\alpha)A + \alpha R^\top R$ that also guarantees the P-matrix property – reduces the iteration count directly. We record the per-step and total complexity of the method against dense factorisation and first-order baselines, and discuss its behaviour on synthetic SPD test problems. The algorithm is used as the computational kernel of the companion papers [25, 24].

1 Introduction

The conjugate gradient method (CG) of Hestenes and Stiefel [12] is the canonical Krylov solver for a symmetric positive definite (SPD) linear system $Ax = b$; see, e.g., Golub and Van Loan [10] or

Greenbaum [11]. It builds its iterates in the Krylov subspace $\mathcal{K}_k(A, b)$ and converges at the $O(\sqrt{\kappa})$ rate, where $\kappa = \kappa(A)$ is the spectral condition number (Proposition 3.1). What CG does *not* do is respect an inequality: it solves equations, not constrained programs. This note is about the gap between the two – how to make CG deliver the solution of the bound-constrained quadratic

$$\min_{x \in \mathbb{R}^n} \; f(x) = \tfrac{1}{2}\, x^\top A x - b^\top x \quad \text{subject to} \quad x \geq 0, \qquad A \succ 0, \tag{1}$$

without giving up its per-iteration cost or its Krylov convergence rate.

Problem (1) is the strictly convex non-negative quadratic program. When $A = M^\top M$ is the Gram matrix of $M \in \mathbb{R}^{m \times n}$ and $b = M^\top d$, its minimiser is the solution of the non-negative least-squares (NNLS) problem $\min_{x \geq 0} \|Mx - d\|_2^2$, the setting of Lawson and Hanson [16]. Without the constraint, the minimiser is simply $x = A^{-1}b$, which is exactly what CG computes; the constraint $x \geq 0$ is what stands between that clean solve and the answer. We also treat the *equality-augmented* variant, in which a linear equality system $Bx = c$ (with $B \in \mathbb{R}^{p \times n}$ of full row rank) is imposed alongside $x \geq 0$, because it arises whenever the solution must satisfy a family of linear balance conditions – the single normalisation $\mathbf{1}^\top x = \beta$ being the case $p = 1$ – and because its multipliers $\lambda \in \mathbb{R}^p$ can be eliminated analytically through a $p \times p$ Schur complement (Section 3).

This note studies the constraint-handling layer directly, and its central contribution is a *termination guarantee*. The vehicle is a *primal-dual active-set* outer loop that wraps CG: it solves the unconstrained system over a working set of *free* variables, releases any variable that returns negative (the primal step), re-admits any bound variable whose reduced gradient is negative (the dual step), and restarts CG on the modified free set each time. The variable toggles are precisely the *principal pivots* of the linear complementarity problem (LCP) $(A, -b)$, and $A \succ 0$ makes it a P-matrix, so every principal submatrix is invertible and each pivot is well defined [18, 14, 15]. Guarding a fast block-pivot path with a least-index Bland-style fallback then yields *unconditional* finite termination at the unique global minimiser, with no non-degeneracy hypothesis (Theorem 5.1) – the hypothesis on which classical block-principal-pivoting termination arguments rely. A perturbation lemma extends the guarantee to the inexact CG solves the loop actually performs: once the residual is small against the problem's decision margin, the inexact loop visits exactly the free sets of the exact one (Lemma 5.1). Integrating this with a Krylov inner solver on a changing free set – restarting CG on each reduced system and warm-starting it across a parametric family of problems – is the engineering that turns the guarantee into a fast solver in practice.

The inner solve preserves two properties that matter for speed. It is *matrix-free*: when $A = M^\top M$, the action $v \mapsto A_{\mathcal{F}} v = M_{\mathcal{F}}^\top (M_{\mathcal{F}} v)$ on the free set $\mathcal{F}$ is evaluated as two products with $M_{\mathcal{F}}$, so A is never assembled and no $O(n^2)$ storage is incurred. And its iteration count is governed by κ, so any preconditioner that compresses the spectrum accelerates it. A ridge/Tikhonov splitting $A \mapsto A_\alpha = (1 - \alpha)A + \alpha R^\top R$ with $R^\top R \succ 0$ does both: it lowers κ (Proposition 6.1) and, by keeping every principal submatrix strictly positive definite, secures the P-matrix property the finite-termination proof relies on (Section 6).

For contrast we analyse a loop-free comparator that enforces the constraint at every step: a *projected-gradient* method that takes a gradient step and projects onto the feasible set – the non-negative orthant for (1), or the simplex slice $\Delta_\beta = \{x : \mathbf{1}^\top x = \beta,\, x \geq 0\}$ for the single-normalisation case, computed in $O(n \log n)$ by a sort-and-threshold pass [9, 6]. It is single-stage and simple, but it is not a Krylov method: it does not orthogonalise residuals against prior search directions and converges at the $O(\kappa)$ rate, a factor $\sqrt{\kappa}$ slower than the CG inner loop (Section 5).

Both building blocks are individually classical – active-set and block-principal-pivoting methods for the non-negative and complementarity problems [16, 14, 15, 18, 20], and CG with preconditioning for SPD systems [12, 10, 11, 2]. What is *not* classical is the termination result: block principal

pivoting is classically shown to terminate only under a non-degeneracy assumption, and the least-index Bland fallback removes it (Theorem 5.1), giving an unconditional guarantee that the synthetic study of Section 7 then shows to be provably necessary yet dormant – the fallback is never entered on generic data. Wrapping this guaranteed loop around a matrix-free Krylov inner solver, warm-started across a parametric sweep, is the engineering that makes the guaranteed method also a fast one. The algorithm is the computational kernel of the companion papers [25, 24], which instantiate it on large structured problems; here it is developed and analysed on its own, independent of any application.

Driving CG through a changing set of free variables has a lineage of its own, and the distinction from it is worth drawing precisely. Bertsekas' projected Newton method [3], Moré and Toraldo's GPCG [17], and Dostál's proportioning and MPRGP algorithms [7, 8] are *feasible-point* methods: every iterate satisfies the bounds, the working face changes through gradient projections, and the switches between projection and CG phases are governed by sufficient-decrease or proportioning tests, with convergence rates in terms of κ and finite identification of the optimal face guaranteed under a non-degeneracy (strict-complementarity) assumption. The loop developed here is instead a *complementary-basis* method: its iterates are infeasible between outer steps, the free set changes by principal pivots driven by signs alone – no line search, no decrease test – and termination is combinatorial, resting on Murty's least-index rule, which is what removes the non-degeneracy hypothesis. On the NNLS side, Bro and De Jong's fast NNLS [4] accelerates Lawson–Hanson by caching normal-equations cross-products, but still assembles $M^\top M$ and solves each working-set system by dense factorisation; the present method forms neither, reaching A only through the two operator applications of Section 4.

Section 2 states the problem and its KKT/complementarity structure; Section 3 reduces it to a sequence of unconstrained SPD solves, records the matrix-free CG operator and its convergence rate, and eliminates the equality multipliers analytically; Section 5 develops the active-set loop and the projected-gradient comparator and proves finite termination; Section 6 treats regularisation and preconditioning; and Section 7 records the complexity of each method and discusses its behaviour on synthetic SPD test problems.

2 Problem and Complementarity Structure

We consider the strictly convex non-negative quadratic program (1),

$$\min_{x\geq 0}\ f(x) = \tfrac{1}{2}\, x^\top A x - b^\top x, \qquad A = A^\top \succ 0, \quad b \in \mathbb{R}^n,$$

together with its two structural specialisations. In the *least-squares* form $A = M^\top M$ and $b = M^\top d$ for $M \in \mathbb{R}^{m\times n}$ of full column rank, so that $f(x) = \frac{1}{2}\|Mx - d\|_2^2 - \frac{1}{2}\|d\|_2^2$ and (1) is NNLS [16]. In the *equality-augmented* form a linear equality system $Bx = c$ is imposed as well,

$$\min_{x}\ \tfrac{1}{2}\, x^\top A x - b^\top x \quad \text{subject to} \quad Bx = c,\ x \geq 0, \qquad B \in \mathbb{R}^{p\times n},\ c \in \mathbb{R}^p, \tag{2}$$

with B of full row rank $p \leq n$, which constrains x to the polytope $\mathcal{P} = \{x \geq 0 : Bx = c\}$, an affine slice of the non-negative orthant. The single normalisation $\mathbf{1}^\top x = \beta$ is the case $p = 1$, $B = \mathbf{1}^\top$, $c = \beta$; the general B carries any finite family of linear balance conditions (group budgets, factor neutralities, netting rules) at the same cost structure.

Optimality and complementarity. Because f is strictly convex and the feasible set of (1) is a non-empty closed convex cone, a unique minimiser exists and the Karush-Kuhn-Tucker (KKT)

conditions are necessary and sufficient. Introducing a multiplier $s \geq 0$ for $x \geq 0$, stationarity of the Lagrangian $\frac{1}{2}x^\top Ax - b^\top x - s^\top x$ gives the reduced gradient $s = \nabla f(x) = Ax - b$, and the KKT system is the linear complementarity problem LCP(A, $-b$):

$$s = Ax - b, \qquad x \geq 0, \qquad s \geq 0, \qquad x_i\, s_i = 0 \;\; \forall i. \tag{3}$$

Complementary slackness partitions the indices into a *free* set $\mathcal{F} = \{i : x_i > 0\}$, on which $s_i = 0$ and hence $(Ax)_i = b_i$, and a *bound* set $\mathcal{B} = \{i : x_i = 0\}$, on which $s_i = (Ax)_i - b_i \geq 0$. A bound variable with $s_i < 0$ would lower f if released to a small positive value; a free variable with $x_i < 0$ is infeasible and must be pushed to the bound. These two violations drive the primal and dual steps of Section 5.

For the equality-augmented problem (2) the Lagrangian carries an additional term $-\lambda^\top(Bx - c)$ with a multiplier $\lambda \in \mathbb{R}^p$, so the reduced gradient becomes $s = Ax - b - B^\top\lambda$ and the free-set stationarity reads $(Ax)_i = b_i + (B^\top\lambda)_i$ for $i \in \mathcal{F}$: on the free set the gradient is pinned to the row space of B, with λ the vector of shadow prices of the p balance conditions. The KKT system is now a *mixed* complementarity problem – the free block carries the equality $B_\mathcal{F} x_\mathcal{F} = c$ alongside the sign conditions. Section 3 shows that λ need not be carried as an unknown in the linear solve; it is recovered in closed form from $p + 1$ SPD solves and a $p \times p$ Schur system (for $p = 1$, the two solves of the single-normalisation case). This requires $B_\mathcal{F}$ to have full row rank, which holds whenever the free set is large enough, $|\mathcal{F}| \geq p$, and generically thereafter; for $p = 1$ it holds for every non-empty free set.

The P-matrix property. Since $A \succ 0$, every principal submatrix $A_\mathcal{F} = A_{\mathcal{F},\mathcal{F}}$ is itself SPD and hence invertible with $\det A_\mathcal{F} > 0$; a matrix all of whose principal minors are positive is a P-matrix [18]. Two consequences follow, both used later. The linear system $A_\mathcal{F}\, x_\mathcal{F} = b_\mathcal{F}$ solved on any candidate free set is well posed, so CG applies at every outer step (Section 3). And LCP(A, $-b$) with a P-matrix has a unique solution reachable by principal pivoting [18, 14], which is the backbone of the finite-termination guarantee in Section 5. When A is only positive semidefinite – as in a rank-deficient least-squares problem, $m < n$ – the regularisation of Section 6 restores $A \succ 0$ and with it the P-matrix property.

3 Reduction to Unconstrained SPD Solves

Fix a candidate free set $\mathcal{F} \subseteq \{1, \ldots, n\}$ and set $x_i = 0$ for $i \notin \mathcal{F}$. On $\mathcal{F}$ the bound-constrained problem (1) reduces to the *unconstrained* strictly convex quadratic

$$\min_{x_\mathcal{F}} \; \tfrac{1}{2}\, x_\mathcal{F}^\top A_\mathcal{F}\, x_\mathcal{F} - b_\mathcal{F}^\top x_\mathcal{F},$$

whose stationarity condition is the SPD linear system

$$A_\mathcal{F}\, x_\mathcal{F} = b_\mathcal{F}. \tag{4}$$

The active-set loop of Section 5 solves a sequence of such systems, one per outer step, and the inequality $x \geq 0$ is imposed by which indices $\mathcal{F}$ contains, never inside the linear solve. The inner solver is CG.

Matrix-free operator. CG accesses $A_{\mathcal{F}}$ only through the mat-vec $v \mapsto A_{\mathcal{F}} v$, never as an explicit matrix. When $A = M^\top M$ this factors as $A_{\mathcal{F}} v = M_{\mathcal{F}}^\top (M_{\mathcal{F}} v)$ with $M_{\mathcal{F}} = M_{:,\mathcal{F}}$: two products with the $m \times |\mathcal{F}|$ submatrix at $O(m\,|\mathcal{F}|)$, so the Gram matrix is never formed and no $O(n^2)$ storage is used – the standard CG-on-the-normal-equations arrangement [10, 21, 5] applied to a column subset that changes between outer steps. This mat-vec is one of the two operations the whole method requires of A; Section 4 isolates that contract and lists the backends that implement it. When a variable is released in the primal step, the operator is rebuilt on the reduced free set and CG restarts on the smaller system.

Proposition 3.1 (CG convergence [11])**.** *Let $A_{\mathcal{F}}$ be SPD with spectral condition number $\kappa = \kappa(A_{\mathcal{F}})$, and let x_k denote the k-th CG iterate for (4). Then*

$$\frac{\|e_k\|_{A_{\mathcal{F}}}}{\|e_0\|_{A_{\mathcal{F}}}} \;\le\; 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k,$$

where $\|e\|_{A_{\mathcal{F}}} = (e^\top A_{\mathcal{F}}\, e)^{1/2}$ is the $A_{\mathcal{F}}$-energy norm of the error $e = x_{\mathcal{F}} - x_k$.

The rate is $O(\sqrt{\kappa})$: reaching a fixed relative energy-norm tolerance takes $O(\sqrt{\kappa}\,\log \varepsilon^{-1})$ iterations. Because $A \succ 0$ forces every $A_{\mathcal{F}} \succ 0$, the bound applies at every outer step, and Section 6 shows how a regularising split lowers κ and hence the count. This $\sqrt{\kappa}$ dependence is the whole reason to keep a Krylov inner solver rather than a plain gradient iteration: the projected-gradient comparator of Section 5 pays $O(\kappa)$ for the same per-step cost.

Eliminating the equality multipliers. For the equality-augmented problem (2), stationarity and feasibility on the free set are the $(|\mathcal{F}|+p) \times (|\mathcal{F}|+p)$ saddle-point system [2]

$$\begin{pmatrix} A_{\mathcal{F}} & -B_{\mathcal{F}}^\top \\ B_{\mathcal{F}} & 0 \end{pmatrix} \begin{pmatrix} x_{\mathcal{F}} \\ \lambda \end{pmatrix} = \begin{pmatrix} b_{\mathcal{F}} \\ c \end{pmatrix}, \qquad B_{\mathcal{F}} = B_{:,\mathcal{F}} \in \mathbb{R}^{p \times |\mathcal{F}|}. \tag{5}$$

Rather than solve the indefinite system directly, eliminate λ by the Schur complement. Stationarity gives $x_{\mathcal{F}} = A_{\mathcal{F}}^{-1}(b_{\mathcal{F}} + B_{\mathcal{F}}^\top \lambda) = v_0 + V_1 \lambda$ in terms of the SPD solves

$$v_0 = A_{\mathcal{F}}^{-1} b_{\mathcal{F}} \in \mathbb{R}^{|\mathcal{F}|}, \qquad V_1 = A_{\mathcal{F}}^{-1} B_{\mathcal{F}}^\top \in \mathbb{R}^{|\mathcal{F}| \times p} \quad \text{(its } p \text{ columns, one CG solve each)}.$$

Imposing $B_{\mathcal{F}} x_{\mathcal{F}} = c$ then fixes the multiplier through the $p \times p$ system

$$S\,\lambda = c - B_{\mathcal{F}} v_0, \qquad S = B_{\mathcal{F}} V_1 = B_{\mathcal{F}} A_{\mathcal{F}}^{-1} B_{\mathcal{F}}^\top \succ 0, \qquad x_{\mathcal{F}} = v_0 + V_1 \lambda,$$

where S, the Schur complement, is SPD because $A_{\mathcal{F}} \succ 0$ and $B_{\mathcal{F}}$ has full row rank; the multiplier is $\lambda = S^{-1}(c - B_{\mathcal{F}} v_0)$ and the $p \times p$ solve is a dense Cholesky, negligible for the small p typical of balance conditions. The single-normalisation case $p = 1$, $B = \mathbf{1}^\top$, $c = \beta$ recovers the two solves $v_0, v_1 = A_{\mathcal{F}}^{-1}\mathbf{1}$ with the scalar $\lambda = (\beta - \mathbf{1}^\top v_0)/(\mathbf{1}^\top v_1)$, and the pure-normalisation case $b = 0$, $\beta = 1$ collapses to the single solve $x_{\mathcal{F}} = v_1/(\mathbf{1}^\top v_1)$. Thus each outer step costs $p+1$ matrix-free CG solves; the multipliers are recovered analytically and never appear in the linear solve, keeping every inner system SPD and directly amenable to Proposition 3.1. The $p+1$ right-hand sides share the operator $A_{\mathcal{F}}$ and can be solved by a block-CG or by independent CG runs, warm-started across outer steps.

4 The operator abstraction

Nothing in Sections 3–5 requires A as an explicit matrix. The active-set loop (Algorithm 1) together with its matrix-free CG inner solve touches A through only two operations:

- a *free-block application* $\text{apply}_{\mathcal{F}}\colon v \mapsto A_{\mathcal{F}} v$ on the current free set, which is all CG needs to solve (4) (and, for the equality-augmented problem, the two right-hand sides of Section 3); and
- a *cross product* $\text{cross}\colon x_{\mathcal{F}} \mapsto A_{\mathcal{B},\mathcal{F}}\, x_{\mathcal{F}}$ from the free block to the bound block, which supplies the reduced gradient $s_i = (Ax)_i - b_i$ at the bound indices $i \in \mathcal{B}$ used by the dual-feasibility test (line 5 of Algorithm 1), since $x_{\mathcal{B}} = 0$.

A full product matvec: $x \mapsto Ax$ is convenient for a residual check but is not otherwise required. The equality-augmented problem (2) adds only the column selections $v \mapsto B_{\mathcal{F}} v$ and $w \mapsto B_{\mathcal{F}}^{\top} w$ used to form the $p+1$ right-hand sides and the Schur complement of Section 3; these act on the given B and are independent of which backend implements A. Capturing exactly this contract as an operator protocol – the same device by which the critical-line algorithm reaches its covariance through a small interface [23, 22] – lets the solver run unchanged on any object that implements it. Several backends do, each realising the two operations at a different cost.

Dense. An explicit SPD $A \in \mathbb{R}^{n\times n}$ implements $\text{apply}_{\mathcal{F}}$ by slicing $A_{\mathcal{F}}$ and cross by the off-diagonal block product. This is the reference backend: matrix-free CG then costs $O(k^2)$ per inner iteration ($k = |\mathcal{F}|$), matching a direct Cholesky solve up to the iteration count, and is worthwhile only when the spectrum makes CG cheap relative to a factorisation. It is the natural choice when A is already assembled and fits in memory.

Gram (data matrix). When $A = M^{\top}M$ for $M \in \mathbb{R}^{m\times n}$ – the least-squares/NNLS setting – the backend implements the protocol straight from M, never forming the $n \times n$ Gram matrix:

$$\text{apply}_{\mathcal{F}}(v) = M_{\mathcal{F}}^{\top}(M_{\mathcal{F}}\, v)\,, \qquad \text{cross}(x_{\mathcal{F}}) = M_{\mathcal{B}}^{\top}(M_{\mathcal{F}}\, x_{\mathcal{F}})\,,$$

each a pair of thin products, at $O(km)$ and $O(nm)$ respectively and $O(nm)$ memory instead of $O(n^2)$. The rank caveat is the familiar one: $M^{\top}M$ has rank at most m, so for $m < n$ the free block becomes singular once $\mathcal{F}$ outgrows the data rank – exactly the degeneracy the regularisation of Section 6 removes.

Factor (diagonal-plus-low-rank). A structured operator

$$A = \text{diag}(d) + U\Delta U^{\top}, \qquad d \in \mathbb{R}^n_{>0},\ U \in \mathbb{R}^{n\times r},\ \Delta \in \mathbb{R}^{r\times r},$$

implements $\text{apply}_{\mathcal{F}}$ and cross through the diagonal and the low-rank factor separately, and inverts the free block by the Woodbury identity

$$A_{\mathcal{F}}^{-1} = D_{\mathcal{F}}^{-1} - D_{\mathcal{F}}^{-1}U_{\mathcal{F}}\, W^{-1}\, U_{\mathcal{F}}^{\top}D_{\mathcal{F}}^{-1}, \qquad W = \Delta^{-1} + U_{\mathcal{F}}^{\top}D_{\mathcal{F}}^{-1}U_{\mathcal{F}} \in \mathbb{R}^{r\times r},$$

at $O(kr^2 + r^3)$ per free-block solve rather than $O(k^3)$, and $O(nr)$ memory. Here the inner solve is *direct* (the Woodbury inverse is exact), so CG is bypassed and the outer loop consumes $A_{\mathcal{F}}^{-1}$ as a black box; the operator protocol is what lets the same active-set loop accept either a matrix-free CG solve or a Woodbury direct solve without modification. This backend is the subject of the second companion paper [24].

Regularised. The ridge split $A_\alpha = (1-\alpha)A + \alpha R^\top R$ of Section 6 is itself a backend: it wraps any of the above and adds the target term to each operation. When $A = M^\top M$ it is the Gram backend of the stacked operator $\tilde{M} = \big(\sqrt{1-\alpha}\, M;\ \sqrt{\alpha}\, R\big)$, so no separate implementation is needed. Because $A_\alpha \succ 0$ strictly, this backend also restores the P-matrix property (Section 2) when the underlying A is only semidefinite.

Because Algorithm 1 and the CG inner solver reach A only through $\text{apply}_\mathcal{F}$ and cross, switching backend – dense to Gram to factor to regularised – requires no change to the loop, the termination proof (Theorem 5.1), or the convergence analysis (Proposition 3.1). The companion papers [25, 24] are, in this light, two backends of one solver.

5 Enforcing Non-Negativity

Section 3 reduces each candidate free set to an SPD solve, leaving $x \geq 0$ as the sole remaining constraint. Two strategies enforce it. The first, an active-set loop, wraps the CG solver unchanged and certifies global optimality through the complementarity system (3); the second, projected gradient, handles the constraint inside every step but forgoes the Krylov rate.

5.1 Active-set / block-principal-pivoting loop

The active-set method maintains a free set $\mathcal{F}$, solves (4) (or its equality-augmented form) on $\mathcal{F}$ by CG, and adjusts $\mathcal{F}$ toward feasibility. In the *primal step* any free variable that returned negative is pushed to the bound and dropped from $\mathcal{F}$. Once all free variables are non-negative, the *dual step* inspects the reduced gradient $s = Ax - b$ (minus $B^\top\lambda$ in the equality-augmented case, with λ from Section 3) at every bound variable: a bound index i with $s_i < 0$ would lower f if released, so it is re-admitted to $\mathcal{F}$. The loop terminates when no variable violates either test, i.e. when $x \geq 0$, $s \geq 0$ and complementary slackness hold to tolerance – exactly the KKT system (3), which for the strictly convex f certifies the unique global minimiser.

Toggling an index between $\mathcal{F}$ and $\mathcal{B}$ is a *principal pivot* on LCP(A, $-b$); dropping or adding several indices at once is a *block* principal pivot [14, 15]. Because $A \succ 0$ is a P-matrix, every principal submatrix is invertible and each pivot is well defined [18]. Block pivots are fast but, like any all-at-once exchange, can cycle: a batch drop can over-shoot the support and a later batch add restore it, revisiting a free set already seen. Algorithm 1 therefore runs the batch exchange as a *fast path* guarded by an anti-cycling counter. Let N be the current number of primal and dual violators and $\bar{N}$ the fewest seen so far. A batch step is taken whenever it makes progress ($N < \bar{N}$) or a patience budget p is unspent; otherwise the algorithm falls back to a single *Bland-style* pivot on the lowest-indexed violator. The least-index single pivot is Murty's principal pivoting method, which cannot cycle [18]; it is what the finite-termination guarantee rests on, the batch path serving only to reach the optimum faster when it does not stall. This is the anti-cycling construction of Júdice and Pires for block principal pivoting [14].

Theorem 5.1 (Finite convergence of Algorithm 1)**.** *Let $f(x) = \frac{1}{2}x^\top Ax - b^\top x$ with $A \succ 0$. Then for any patience budget $p_{\max} \in \mathbb{N}$, Algorithm 1 terminates in finitely many outer iterations and returns the unique global minimiser of $\min_{x\geq 0} f(x)$. No non-degeneracy assumption is required. The same holds for the equality-augmented problem (2) with the reduced gradient of line 5 adjusted by $B^\top\lambda$, provided $B_\mathcal{F}$ retains full row rank on every free set the loop visits (automatic for $p = 1$; generic once $|\mathcal{F}| \geq p$).*

Proof. Step 1 (each inner solve is well posed). Because $A \succ 0$, every principal submatrix $A_\mathcal{F}$ is SPD, so A is a P-matrix [18], and the system $A_\mathcal{F}x_\mathcal{F} = b_\mathcal{F}$ has a unique solution to which CG converges

Algorithm 1 Active-set outer loop (wraps the CG inner solver f)

Require: Inner solver f returning $(x_{\mathcal{F}}, k_{\text{step}})$ for free set $\mathcal{F}$; SPD A (or its mat-vec) and b; tolerance ε; patience $p_{\max} \in \mathbb{N}$ *(default 3)*

Ensure: Minimiser $x \in \mathbb{R}^n$ of (1); outer steps s; total inner iterations k

$\mathcal{F} \leftarrow \{1, \dots, n\}$; $s \leftarrow 0$; $k \leftarrow 0$; $\bar{N} \leftarrow n+1$; $p \leftarrow p_{\max}$
loop
  $(x_{\mathcal{F}}, k_{\text{step}}) \leftarrow f(\mathcal{F})$; $s \leftarrow s+1$; $k \leftarrow k + k_{\text{step}}$ *(k_{step}: CG iterations; 1 for a direct inner solve)*
  $x_i \leftarrow (x_{\mathcal{F}})_i$ for $i \in \mathcal{F}$; $x_i \leftarrow 0$ for $i \notin \mathcal{F}$
  $s_i \leftarrow (Ax)_i - b_i$ *(reduced gradient; subtract $(B^\top \lambda)_i$ in the equality-augmented case)*
  $\mathcal{D} \leftarrow \{i \in \mathcal{F} : x_i < -\varepsilon\}$; $\mathcal{V} \leftarrow \{i \notin \mathcal{F} : s_i < -\varepsilon\}$ *(primal and dual violators)*
  $\mathcal{H} \leftarrow \mathcal{D} \cup \mathcal{V}$; $N \leftarrow |\mathcal{H}|$
  if $N = 0$ **then**
    break *(KKT (3) satisfied; globally optimal)*
  else if $N < \bar{N}$ **or** $p > 0$ **then** *(fast path: progress, or patience remains)*
    if $N < \bar{N}$ **then** $\bar{N} \leftarrow N$; $p \leftarrow p_{\max}$ **else** $p \leftarrow p - 1$
    $\mathcal{F} \leftarrow (\mathcal{F} \setminus \mathcal{D}) \cup \mathcal{V}$ *(batch exchange: drop all $\mathcal{D}$, add all $\mathcal{V}$)*
  else *(anti-cycling fallback: p exhausted)*
    $i^\star \leftarrow \min\{i : i \in \mathcal{H}\}$ *(Bland least-index rule)*
    if $i^\star \in \mathcal{D}$ **then** $\mathcal{F} \leftarrow \mathcal{F} \setminus \{i^\star\}$ **else** $\mathcal{F} \leftarrow \mathcal{F} \cup \{i^\star\}$ *(single principal pivot)*
  end if
end loop
return x, s, k

(Proposition 3.1). In the equality-augmented case the free-set solve is the saddle system (5); with $A_{\mathcal{F}} \succ 0$ and $B_{\mathcal{F}}$ of full row rank its Schur complement S is SPD, so (5) has the unique solution recovered by the $p+1$ SPD solves of Section 3.

Step 2 (termination implies global optimality). Free-set stationarity gives $s_i = 0$ for $i \in \mathcal{F}$. The loop exits only when $N = 0$: (i) all free $x_i \geq 0$ and (ii) all bound $s_i \geq 0$. These are exactly $x \geq 0$, $s \geq 0$, $x_i s_i = 0$ – the KKT system (3). Strict convexity of f makes them sufficient for the unique global minimiser; it therefore suffices to show the loop cannot run forever.

Step 3 (the fallback cannot cycle). Call a maximal block of consecutive iterates taking the **else** branch (the single Bland pivot) a *fallback run*. Within a run $\bar{N}$ is constant, and each iterate performs one principal pivot on the lowest-indexed violator. This is precisely Murty's least-index principal pivoting method for the P-matrix LCP of Step 2, which generates a sequence of complementary bases (here, free sets) in which no basis recurs and which reaches the solution in finitely many pivots [18]. A fallback run is an initial segment of that sequence started from the free set left by the preceding step, so each run is finite, wherever it starts.

Step 4 (only finitely many batch steps and runs). The counter satisfies $0 \leq N \leq n$, and $\bar{N}$ is non-increasing, strictly decreasing exactly when the progress branch ($N < \bar{N}$) fires; hence that branch fires at most $n+1$ times. Each remaining fast-path step has $N \geq \bar{N}$ and decrements p, and p resets only on a progress step, so at most $p_{\max}$ such steps occur between progress events. The number of fallback runs is likewise bounded: a run ends at termination ($N = 0$) or at a progress step, and there are at most $n+1$ progress steps. Thus the algorithm takes at most $(n+1) + (n+2)\, p_{\max}$ fast-path steps and at most $n+2$ fallback runs.

Step 5 (finite termination). By Step 3 each of the at most $n+2$ fallback runs is finite, and by Step 4 the fast-path steps are finite in number; their sum is finite, so the loop reaches $N = 0$ after

finitely many outer iterations, and Step 2 certifies the returned x as the unique global minimiser. A crude ceiling is the number of complementary bases, 2^n. □

The 2^n ceiling establishes *finiteness* only; it is not an efficiency estimate. The guarantee is unconditional: the least-index fallback removes the non-degeneracy hypothesis that earlier active-set arguments need, and it is the fallback – not the batch exchange – that the proof rests on. In practice the fallback is rarely reached, because a batch step fails to reduce N only when a dropped index is immediately re-added (or vice versa), which forces $s_i = 0$ exactly for some toggled i – a codimension-one, hence non-generic, condition [14]; block principal pivoting therefore converges in a handful of outer steps on typical data, with the fast path alone certifying optimality.

Theorem 5.1 assumes each inner call returns the exact minimiser of $f_{\mathcal{F}}$; CG returns an iterate accurate only to its residual tolerance. The following lemma closes that gap: once the residual is tight enough relative to the test threshold ε, every primal and dual sign decision of Algorithm 1 coincides with the decision the exact solve would make, so the inexact loop inherits the theorem wholesale.

Lemma 5.1 (Inexact inner solves)**.** *For a free set $\mathcal{F}$ let $\hat{x}(\mathcal{F})$ denote the exact solution of (4) extended by zeros to $\mathbb{R}^n$, and $\hat{s}(\mathcal{F}) = A\,\hat{x}(\mathcal{F}) - b$ its reduced gradient. Define the* decision margin

$$\mu \;=\; \min_{\mathcal{F}\subseteq\{1,\dots,n\}} \min\Big(\big\{\,|\hat{x}_i(\mathcal{F})| : i\in\mathcal{F},\ \hat{x}_i(\mathcal{F})\neq 0\,\big\} \cup \big\{\,|\hat{s}_i(\mathcal{F})| : i\notin\mathcal{F},\ \hat{s}_i(\mathcal{F})\neq 0\,\big\}\Big),$$

the smallest nonzero magnitude any primal or dual test can encounter; $\mu > 0$ as a minimum of finitely many positive numbers. Suppose the test tolerance satisfies $0 < \varepsilon < \mu$ and every inner CG call is stopped at a residual $\rho = \|b_{\mathcal{F}} - A_{\mathcal{F}}\tilde{x}_{\mathcal{F}}\|_2$ obeying

$$\rho\,\frac{1+\lambda_{\max}(A)}{\lambda_{\min}(A)} \;<\; \min\big(\varepsilon,\ \mu-\varepsilon\big). \tag{6}$$

Then at every outer step the violator sets $\mathcal{D}, \mathcal{V}$ of Algorithm 1 computed from the CG iterate coincide with those computed from the exact solve on the same free set. The inexact loop therefore traverses exactly the free-set sequence of the exact loop, Theorem 5.1 applies verbatim, and the returned iterate satisfies $\|\tilde{x} - x^\star\|_\infty \le \rho/\lambda_{\min}(A)$.

Proof. Write $r = b_{\mathcal{F}} - A_{\mathcal{F}}\tilde{x}_{\mathcal{F}}$, so $\tilde{x}_{\mathcal{F}} - \hat{x}_{\mathcal{F}} = -A_{\mathcal{F}}^{-1}r$. Cauchy interlacing gives $\lambda_{\min}(A_{\mathcal{F}}) \ge \lambda_{\min}(A)$, hence

$$\delta_x \;:=\; \|\tilde{x}_{\mathcal{F}} - \hat{x}_{\mathcal{F}}\|_\infty \;\le\; \|A_{\mathcal{F}}^{-1}r\|_2 \;\le\; \rho/\lambda_{\min}(A).$$

For a bound index $i \notin \mathcal{F}$ the reduced gradients differ by $\tilde{s}_i - \hat{s}_i = A_{i,\mathcal{F}}(\tilde{x}_{\mathcal{F}} - \hat{x}_{\mathcal{F}})$, and $\|A_{i,\mathcal{F}}\|_2 \le \|A\|_2 = \lambda_{\max}(A)$, so $\delta_s := \max_i |\tilde{s}_i - \hat{s}_i| \le \lambda_{\max}(A)\,\rho/\lambda_{\min}(A)$. Both perturbations are bounded by $\delta := \rho\,(1+\lambda_{\max}(A))/\lambda_{\min}(A) < \min(\varepsilon, \mu-\varepsilon)$ by (6).

Fix $i \in \mathcal{F}$. If $\hat{x}_i \ge 0$ then $\hat{x}_i = 0$ or $\hat{x}_i \ge \mu$; either way $\tilde{x}_i \ge -\delta > -\varepsilon$, so the inexact test does not flag i, matching the exact test. If $\hat{x}_i < 0$ then $\hat{x}_i \le -\mu$ by the definition of μ, so $\tilde{x}_i \le -\mu + \delta < -\varepsilon$: both tests flag i. The identical case split with $\hat{s}_i$ and δ_s handles the dual test. Hence $\tilde{\mathcal{D}} = \hat{\mathcal{D}}$ and $\tilde{\mathcal{V}} = \hat{\mathcal{V}}$ at this step; since both loops start from $\mathcal{F} = \{1,\dots,n\}$ and the counters $\bar{N}, p$ are functions of the violator sequence, induction gives the same free-set trajectory and the same stopping step. At termination the exact iterate is $x^\star$ (Theorem 5.1), so $\|\tilde{x} - x^\star\|_\infty = \delta_x \le \rho/\lambda_{\min}(A)$, the off-set entries being zero in both. □

Remark 5.1 (Reading the margin condition)**.** *Three comments. (i) Components that are exactly zero at an exact solve are* not *excluded: the threshold ε absorbs them on both sides of the comparison,*

so the lemma – like the theorem it protects – needs no non-degeneracy assumption. Moreover, since $\mu > \varepsilon$, the ε-thresholded tests of Algorithm 1 agree with the exact sign tests of the pivot argument in Theorem 5.1, so the algorithm as implemented is the algorithm as analysed. (ii) When CG is stopped at a relative *residual $\varepsilon_{\mathrm{cg}}$, then $\rho \le \varepsilon_{\mathrm{cg}}\|b\|_2$ and condition (6) becomes a bound on $\varepsilon_{\mathrm{cg}}$ alone. (iii) The margin μ is not computable a priori – the lemma is a license, not a recipe: it guarantees that* some *finite tolerance reproduces the exact trajectory, and in practice a residual tolerance a couple of orders of magnitude below ε suffices (Section 7.2 verifies trajectory-for-trajectory agreement at $\varepsilon_{\mathrm{cg}} = 10^{-10}$, $\varepsilon = 10^{-8}$).*

5.2 Projected-gradient comparator

The second strategy enforces the constraint at every step. Projected gradient iterates

$$x_{k+1} = \Pi_C(x_k - \tau\,\nabla f(x_k))\,, \qquad \nabla f(x) = Ax - b,$$

where C is the feasible set. For (1) it is the non-negative orthant, so $\Pi_C(y) = \max(y, 0)$ componentwise; for the single normalisation $\mathbf{1}^\top x = \beta$, $x \ge 0$ it is the simplex slice Δ_β, projected onto in $O(n \log n)$ by a single sort-and-threshold pass [9, 6]. For a general equality system $Bx = c$, however, the projection onto $\mathcal{P} = \{x \ge 0 : Bx = c\}$ is itself a non-negative quadratic program with no closed form, so projected gradient loses its per-step simplicity – one reason the active-set loop, whose cost rises only by the p extra right-hand sides of Section 3, is the method of choice once $p > 1$. The step size is $\tau = 1/L$ with $L = \lambda_{\max}(A)$, estimated once by power iteration on the same (matrix-free) operator. There is no outer loop and no linear solve: each step is one gradient evaluation – two products with M when $A = M^\top M$ – plus a projection.

Proposition 5.1 (Projected-gradient convergence [19])**.** *Let $A \succ 0$, so f is μ-strongly convex and L-smooth with $\mu = \lambda_{\min}(A)$ and $L = \lambda_{\max}(A)$, and let $x^\star = \arg\min_{x \in C} f(x)$. Projected gradient with $\tau = 1/L$ satisfies*

$$\|x_k - x^\star\|^2 \;\le\; \Big(1 - \tfrac{1}{\kappa(A)}\Big)^k \|x_0 - x^\star\|^2, \qquad k \ge 0.$$

Proof sketch. $x^\star = \Pi_C(x^\star - \tau\nabla f(x^\star))$ is a fixed point; non-expansiveness of Π_C and co-coercivity of the L-smooth gradient give $\|x_{k+1} - x^\star\|^2 \le \|x_k - x^\star\|^2 - \tau(2 - \tau L)\langle \nabla f(x_k) - \nabla f(x^\star),\, x_k - x^\star\rangle$. Setting $\tau = 1/L$ makes $2 - \tau L = 1$, and μ-strong convexity bounds the inner product below by $\mu\|x_k - x^\star\|^2$, so $\|x_{k+1} - x^\star\|^2 \le (1 - \mu/L)\|x_k - x^\star\|^2$. □

The iteration count scales as $O(\kappa)$, against $O(\sqrt{\kappa})$ for the CG inner loop (Proposition 3.1). Both methods pay the same per-step cost – one operator application – so the $\sqrt{\kappa}$ gap in iteration counts is a runtime gap on ill-conditioned problems. Nesterov acceleration would lift the projected-gradient rate to $O(\sqrt{\kappa})$, but CG attains that rate *optimally* within the Krylov subspace: each iterate minimises f over $\mathcal{K}_k(A_{\mathcal{F}}, b_{\mathcal{F}})$, a strictly richer search space than two-point momentum, and typically with a smaller constant. Plain projected gradient is therefore the honest first-order comparator, isolating the Krylov advantage; a FISTA variant [1] narrows the constant but not the asymptotic gap. The trade-off is otherwise in the other direction: projected gradient is loop-free and needs no complementarity certificate, so it is the simpler method when κ is small or moderate accuracy suffices.

5.3 Warm-starting

For a parametric family of problems – e.g. a sequence $b(\theta_1), b(\theta_2), \dots$ of right-hand sides, or a swept normalisation β – consecutive minimisers usually differ in only a few active constraints. Passing the previous problem's $(\mathcal{F}, x)$ pair as the initial state of the next solve reduces both the outer count (the free set needs fewer primal and dual adjustments) and the inner count (CG starts from a small initial residual, its guess being the previous solution restricted to the new free set and renormalised). A direct inner solver profits only from the warm free set, having no analogue of an initial guess; the matrix-free CG inner solver profits from both, which is where the parametric sweeps of the companion papers [25, 24] obtain their largest speed-ups.

6 Regularisation and Preconditioning

Both the inner CG rate (Proposition 3.1) and the projected-gradient rate (Proposition 5.1) are governed by $\kappa = \kappa(A)$, so anything that compresses the spectrum of A shortens the solve. Two mechanisms are available, and they coincide in a convenient way.

Regularising split. Replace A by the convex combination

$$A_\alpha = (1-\alpha)\,A + \alpha\,R^\top R, \qquad \alpha \in (0,1),\ R^\top R \succ 0, \tag{7}$$

a ridge/Tikhonov regularisation of A toward the SPD target $R^\top R$. When $A = M^\top M$ this is again a Gram matrix, of the stacked operator

$$\tilde{M} = \begin{pmatrix} \sqrt{1-\alpha}\,M \\ \sqrt{\alpha}\,R \end{pmatrix}, \qquad \tilde{M}^\top \tilde{M} = A_\alpha,$$

so it is simply the Gram backend of $\tilde{M}$ (the regularised operator of Section 4) and CG runs on $\tilde{M}_\mathcal{F}$ with no change. The augmented-matrix device is standard in least-squares regularisation [10, 16].

Proposition 6.1 (Condition number under the regularising split)**.** *For A_α as in (7), Weyl's inequality [13] gives*

$$\kappa(A_\alpha) \;\le\; \frac{(1-\alpha)\lambda_{\max}(A) + \alpha\,\lambda_{\max}(R^\top R)}{\alpha\,\lambda_{\min}(R^\top R)}.$$

The bound is strictly decreasing in α; as $\alpha \to 1$ it approaches $\kappa(R^\top R)$, which equals 1 for $R = \sqrt{c}\,I$. For the scaled-identity target $R^\top R = \bar{\lambda} I$ it simplifies to $\kappa(A_\alpha) \le \frac{1-\alpha}{\alpha}\,\frac{\lambda_{\max}(A)}{\bar{\lambda}} + 1 = O\big((1-\alpha)/\alpha\big)$.

The split does double duty. It lowers κ and hence the iteration count of both solvers (Propositions 3.1 and 5.1), and – because $A_\alpha \succ 0$ strictly whenever $\alpha > 0$, even when A is only positive semidefinite (a rank-deficient least-squares operator with $m < n$) – it restores the P-matrix property on which Theorem 5.1 depends. A strictly positive α is therefore the natural setting for the active-set loop: it simultaneously conditions the inner solve and guarantees that every principal submatrix is invertible.

Preconditioning. When the operator must be solved at its given A, a symmetric positive definite preconditioner $P \approx A$ replaces the CG rate's $\kappa(A)$ by $\kappa(P^{-1}A)$; standard choices (Jacobi, incomplete Cholesky, or a low-rank-plus-diagonal approximation built from a few dominant eigenpairs) apply unchanged inside the active-set loop, since preconditioned CG still accesses $A_\mathcal{F}$ only through its mat-vec [10, 2]. The regularising split (7) may itself be read as an implicit preconditioner: it changes

the problem, but toward a target the application deems statistically or structurally preferable, so the conditioning gain comes at no modelling cost. The companion papers [25, 24] develop concrete instances – a scaled-identity split and a low-rank-plus-identity target inverted by the Woodbury identity – and quantify the resulting iteration savings on their structured operators.

7 Complexity and Numerical Behaviour

7.1 Complexity

Table 1 collects the asymptotic cost of solving (1) by each method, for $A = M^\top M$ with $M \in \mathbb{R}^{m\times n}$, condition number $\kappa = \kappa(A_\alpha)$ after the regularising split, tolerance ε, free-set size $k = |\mathcal{F}| \le n$, and s active-set outer steps. "Extra memory" is storage beyond the operator; the matrix-free rows never form A and need only $O(n)$ working memory.

Method	Per-step cost	Iterations	Extra memory	Forms A
Interior point (dense QP)	$O(n^3)$	$O(\log 1/\varepsilon)$	$O(n^2)$	Yes
Dense Cholesky + active set	$O(k^2 m)$	s	$O(k^2)$	Yes
CG matrix-free + active set	$O(km)$	$s \cdot O(\sqrt{\kappa})$	$O(n)$	No
Factor / Woodbury + active set	$O(kr^2 + r^3)$	s	$O(nr)$	No
Projected gradient (matrix-free)	$O(nm)$	$O(\kappa)$	$O(n)$	No

Table 1: Asymptotic cost of solving the non-negative quadratic (1) with $A = M^\top M$, $M \in \mathbb{R}^{m\times n}$. Here $k = |\mathcal{F}| \le n$ is the free-set size, s the number of active-set outer steps, $\kappa = \kappa(A_\alpha)$ the condition number after the split (7), and r the factor rank of a diagonal-plus-low-rank operator (Section 4). The last four rows share the active-set loop and differ only in the backend (Section 4): matrix-free CG and the Woodbury factor solve both avoid assembling A, the former iteratively at $O(\sqrt{\kappa})$ inner steps, the latter by an exact rank-r inverse. The active-set rows differ only in the inner solve: matrix-free CG applies $M_{\mathcal{F}}^\top(M_{\mathcal{F}} v)$ at $O(km)$ and never assembles the Gram matrix, whereas dense Cholesky forms and factorises $A_{\mathcal{F}}$ at $O(k^2 m)$; the matrix-free advantage grows with k. The CG "Iterations" column counts $s \cdot O(\sqrt{\kappa})$ inner iterations in total; because the loop starts at $\mathcal{F} = \{1, \dots, n\}$, the first solve at $k = n$ dominates and is governed by the full κ bounded in Proposition 6.1. Projected gradient avoids the linear solve entirely but pays $O(\kappa)$, a factor $\sqrt{\kappa}$ more inner iterations than CG at the same per-step cost.

Two comparisons are structural. Within the active-set family, matrix-free CG and dense Cholesky share the outer loop and differ only in whether $A_{\mathcal{F}}$ is assembled; the $O(km)$-versus-$O(k^2 m)$ gap means CG pulls ahead as the free set grows, while dense Cholesky can win when k is small or κ is large enough that CG needs many inner iterations (the crossover is near $\kappa \sim k^2$). Between the two matrix-free methods, CG's $O(\sqrt{\kappa})$ dominates projected gradient's $O(\kappa)$ whenever κ is large; only when the regularising split compresses κ to $O(1)$ do the inner counts become comparable, at which point projected gradient's loop-free simplicity is attractive.

The equality-augmented problem (2) multiplies the inner cost of an outer step by the $p + 1$ right-hand sides that share the operator $A_{\mathcal{F}}$ (Section 3), plus an $O(p^3)$ dense Schur solve that is negligible for the small p of typical balance conditions; the asymptotics in n, m, and κ are otherwise those of Table 1.

7.2 Behaviour on synthetic SPD problems

We test the propositions on a controlled family with a known optimum. Take $A = Q\Lambda Q^\top$ with Q a random orthogonal matrix and a geometric spectrum $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$ on $[1, \kappa]$ of prescribed condition number κ (equivalently $A = M^\top M$ for a random M with the chosen singular values), and plant a non-negative solution $x^\star \geq 0$ of a chosen support by setting $b = Ax^\star - s^\star$ for a complementary $s^\star \geq 0$; then $(x^\star, s^\star)$ is the exact solution of $\mathrm{LCP}(A, -b)$ in closed form. The runs below use $n = 200$ (support 50%) averaged over five seeds, with the smaller $n = 120$ where the projected-gradient $O(\kappa)$ cost forces a capped κ range; all are reproduced by a self-contained NumPy script that implements Algorithm 1, the CG inner solve, and the comparator directly from their definitions. The geometric spectrum is the honest choice: its clustering lets CG converge faster than the worst-case bound, so the measured growth is an *upper*-bounded confirmation of Proposition 3.1, not a fitted exponent.

κ	Outer steps s	CG inner (total)	Fallback pivots
10^1	3.0	85	0.00
10^2	3.2	212	0.00
10^3	4.2	541	0.00
10^4	5.0	1180	0.00
10^5	5.2	2294	0.00
10^6	6.2	4529	0.00

Table 2: Active-set solve of (1) on the synthetic family ($n = 200$, support 50%, mean over five seeds). Outer steps stay in single digits and grow only weakly with κ; the total CG inner count grows sublinearly in κ (slope 0.35 in log–log, $R^2 = 0.995$, against the $O(\sqrt{\kappa})$ worst case); the least-index Bland fallback is never triggered on this generic data, so the fast block-pivot path certifies optimality by itself. The solver recovers the planted $x^\star$ to $7e-10$.

Three predictions of the analysis are borne out.

(i) Inner count is bounded by $\sqrt{\kappa}$; regularisation lowers it. Figure 1 shows the total CG inner iterations rising with κ but staying under the $O(\sqrt{\kappa})$ envelope of Proposition 3.1 (measured slope $0.35 < \frac{1}{2}$, the clustered spectrum buying CG its superlinear phase). The regularising split (7) with $R^\top R = I$ compresses κ at rate $O((1-\alpha)/\alpha)$ (Proposition 6.1) and drives the count down monotonically as α increases (Figure 3); the effect is largest in the rank-deficient regime $m < n$, where only $\alpha > 0$ makes the system well posed (Table 3 below).

(ii) Outer count is small and the fallback is dormant. Across $\kappa \in [10, 10^6]$ the active-set loop terminates in at most 6 outer steps (Table 2), adjusting the free set by a few indices per step and reaching the correct support in a count that tracks its distance from the initial full set, not n. The patience budget is never exhausted – the Bland fallback of Theorem 5.1 guarantees termination but is not entered on generic data, exactly the codimension-one argument of Section 5.

(iii) CG beats projected gradient by a widening $\sqrt{\kappa}$ factor. Figure 2 plots both matrix-free methods at matched per-step cost. Both run faster than their worst-case bounds on the clustered spectrum, but projected gradient's growth exponent is about twice CG's, so their ratio grows as $\kappa^{0.38}$ – essentially the predicted $\sqrt{\kappa}$ – from parity at $\kappa = 10$ to $16\times$ at $\kappa = 10^4$. This is the $O(\sqrt{\kappa})$-versus-$O(\kappa)$ separation of Propositions 3.1 and 5.1, widening with κ and closing under heavy regularisation.

Rank deficiency. Table 3 probes the semidefinite regime the regularising split is claimed to repair: a Gram operator $A = M^\top M$ with $M \in \mathbb{R}^{100\times 200}$, so every free block with $|\mathcal{F}| > m = 100$ is singular – including the initial one, since the loop starts from $\mathcal{F} = \{1, \ldots, n\}$. The failure at $\alpha = 0$ is instructive in its anatomy. When the planted support lies *below* the rank, the first inner solve

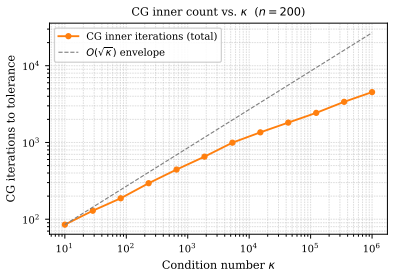

Figure 1: CG inner count vs. κ, under the $O(\sqrt{\kappa})$ envelope.

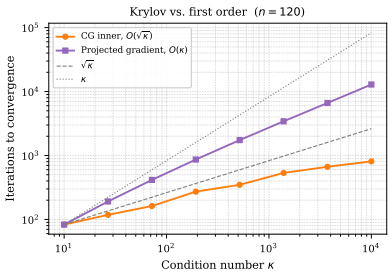

Figure 2: Krylov vs. first order: $O(\sqrt{\kappa})$ vs. $O(\kappa)$.

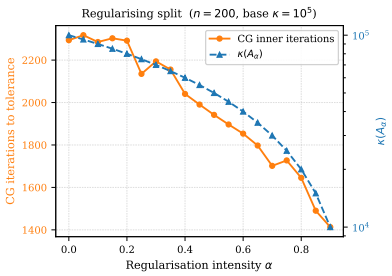

Figure 3: Regularising split: count falls as $\kappa(A_\alpha)$ shrinks.

can never meet its tolerance (the residual stalls at the component of $b_{\mathcal{F}}$ outside the range of $A_{\mathcal{F}}$, so the iteration cap is hit and the iterate carries no certificate), yet the uncertified signs may still identify the support, after which the shrunken free blocks are regular: the loop then reaches the optimum on 4 of five seeds – at an order of magnitude more inner iterations – and fails to converge on the remainder. When the optimal support *exceeds* the rank, the final free block is itself singular, no certificate is ever available, and the loop converges on 0 of five seeds. Any $\alpha > 0$ restores the P-matrix premise of Theorem 5.1, and with it both reliability and speed: every run converges, recovering the planted optimum to $1e-09$. The point is sharpened rather than softened by the partial successes at $\alpha = 0$: what regularisation buys is not merely conditioning but the *guarantee* – without it, termination is luck.

Support k	α	Converged	CG inner	Capped solves	$\|x - x^\star\|_\infty$
50	0.00	4/5	2716	5	$1 \cdot 10^{-9}$
50	0.05	5/5	190	0	$6 \cdot 10^{-10}$
50	0.20	5/5	118	0	$4 \cdot 10^{-10}$
150	0.00	0/5	–	5	–
150	0.05	5/5	351	0	$1 \cdot 10^{-9}$
150	0.20	5/5	163	0	$7 \cdot 10^{-10}$

Table 3: The rank-deficient regime ($n = 200$, $m = 100$, five seeds; CG capped at 2000 iterations per solve, 30 outer steps). "Capped solves" counts runs whose first inner solve hit the iteration cap without meeting its tolerance. At $\alpha = 0$ the loop is unreliable below the rank and fails uniformly above it; any $\alpha > 0$ converges on every seed with an order of magnitude fewer inner iterations.

Preconditioning. Section 6's claim that a preconditioner slots into the loop unchanged is tested on operators with a well-conditioned core and a badly scaled diagonal, $A = D^{1/2}(Q\Lambda Q^\top)D^{1/2}$ with $\kappa(Q\Lambda Q^\top) = 10^2$ and the entries of D spread over four decades. Jacobi-preconditioned CG inside Algorithm 1 runs at the core's conditioning regardless of the spread (326 inner iterations against 243 for the unscaled operator), while plain CG pays for the scaling in full (2983 iterations at $\kappa(A) \approx 9e+04$): a factor of 9 recovered by a preconditioner that costs one vector division per iteration.

Inexact inner solves. In line with Lemma 5.1, running the loop with its CG inner solver ($\varepsilon_{\text{cg}} = 10^{-10}$, $\varepsilon = 10^{-8}$) against a twin loop whose inner solver is an exact direct solve produces *identical* free-set trajectories on all 15 planted instances ($\kappa \in \{10^2, 10^4, 10^6\}$, five seeds each): the guarantee of Theorem 5.1 survives the inexact solves it is implemented with, not merely in principle but decision-for-decision.

The general equality constraint. The same loop with the Schur-complement elimination of Section 3 solves the equality-augmented problem (2) for a general $Bx = c$. On planted instances with $B \in \mathbb{R}^{p\times n}$ of full row rank, the solver recovers the optimum to $9e-09$ and meets the equality to machine precision for $p \in \{1, 3, 10\}$, with the inner cost scaling as the $p+1$ shared right-hand sides anticipated in Section 3; the case $p = 1$ reproduces the single-normalisation solve exactly.

Quantitative wall-clock studies on structured operators arising in applications – including scaling with problem size and warm-started parametric sweeps – are reported in the companion papers [25, 24], whose numerical sections instantiate Algorithm 1 with the matrix-free CG inner solver analysed here.

8 Conclusions

Conjugate gradients solves SPD systems; it does not, unaided, respect $x \geq 0$. This note supplies the missing layer. The bound-constrained quadratic $\min_{x\geq 0} \frac{1}{2}x^\top Ax - b^\top x$ – and its least-squares and equality-augmented ($Bx = c$) variants – reduces, on any fixed free set, to an unconstrained SPD solve that CG performs matrix-free; non-negativity is enforced around it by a primal-dual active-set loop whose asset toggles are the principal pivots of $\mathrm{LCP}(A, -b)$. Because $A \succ 0$ is a P-matrix, guarding a fast block-pivot path with a least-index Bland fallback yields unconditional finite termination at the unique global minimiser (Theorem 5.1), with no non-degeneracy assumption. The inner solver retains CG's $O(\sqrt{\kappa})$ Krylov rate, a factor $\sqrt{\kappa}$ ahead of the loop-free projected-gradient comparator, and a regularising split $A_\alpha = (1-\alpha)A + \alpha R^\top R$ simultaneously compresses κ and secures the P-matrix property, so it both accelerates the inner solve and licenses the termination proof.

The construction is deliberately application-agnostic: it takes an SPD operator (or a rectangular M with $A = M^\top M$), a right-hand side, and an optional linear equality system $Bx = c$, and returns the non-negative minimiser. The companion papers [25, 24] instantiate it as their computational kernel and report quantitative benchmarks on structured operators; the guarantees and complexity recorded here hold for any SPD A.

References

[1] A. Beck and M. Teboulle, *A fast iterative shrinkage-thresholding algorithm for linear inverse problems*, SIAM Journal on Imaging Sciences, 2 (2009), pp. 183–202.

[2] M. Benzi, G. H. Golub, and J. Liesen, *Numerical solution of saddle point problems*, Acta Numerica, 14 (2005), pp. 1–137.

[3] D. P. Bertsekas, *Projected Newton methods for optimization problems with simple constraints*, SIAM Journal on Control and Optimization, 20 (1982), pp. 221–246.

[4] R. Bro and S. De Jong, *A fast non-negativity-constrained least squares algorithm*, Journal of Chemometrics, 11 (1997), pp. 393–401.

[5] S.-C. T. Choi, *Iterative Methods for Singular Linear Equations and Least-Squares Problems*, phd thesis, Stanford University, 2006.

[6] L. Condat, *Fast projection onto the simplex and the ℓ_1 ball*, Mathematical Programming, 158 (2016), pp. 575–585.

[7] Z. Dostál, *Box constrained quadratic programming with proportioning and projections*, SIAM Journal on Optimization, 7 (1997), pp. 871–887.

[8] Z. Dostál and J. Schöberl, *Minimizing quadratic functions subject to bound constraints with the rate of convergence and finite termination*, Computational Optimization and Applications, 30 (2005), pp. 23–43.

[9] J. Duchi, S. Shalev-Shwartz, Y. Singer, and T. Chandra, *Efficient projections onto the ℓ_1-ball for learning in high dimensions*, in Proceedings of the 25th International Conference on Machine Learning (ICML), New York, NY, 2008, ACM, pp. 272–279.

[10] G. H. Golub and C. F. Van Loan, *Matrix Computations*, Johns Hopkins University Press, 4th ed., 2013.

[11] A. Greenbaum, *Iterative Methods for Solving Linear Systems*, vol. 17 of Frontiers in Applied Mathematics, SIAM, Philadelphia, 1997.

[12] M. R. Hestenes and E. Stiefel, *Methods of conjugate gradients for solving linear systems*, Journal of Research of the National Bureau of Standards, 49 (1952), pp. 409–436.

[13] R. A. Horn and C. R. Johnson, *Matrix Analysis*, Cambridge University Press, 2nd ed., 2013.

[14] J. J. Júdice and F. M. Pires, *A block principal pivoting algorithm for large-scale strictly monotone linear complementarity problems*, Computers & Operations Research, 21 (1994), pp. 587–596.

[15] J. Kim and H. Park, *Fast nonnegative matrix factorization: An active-set-like method and comparisons*, SIAM Journal on Scientific Computing, 33 (2011), pp. 3261–3281.

[16] C. L. Lawson and R. J. Hanson, *Solving Least Squares Problems*, Prentice-Hall, Englewood Cliffs, NJ, 1974.

[17] J. J. Moré and G. Toraldo, *On the solution of large quadratic programming problems with bound constraints*, SIAM Journal on Optimization, 1 (1991), pp. 93–113.

[18] K. G. Murty, *Linear Complementarity, Linear and Nonlinear Programming*, vol. 3 of Sigma Series in Applied Mathematics, Heldermann Verlag, Berlin, 1988.

[19] Y. Nesterov, *Introductory Lectures on Convex Optimization: A Basic Course*, Kluwer Academic Publishers, Boston, MA, 2004.

[20] J. Nocedal and S. J. Wright, *Numerical Optimization*, Springer, New York, 2nd ed., 2006.

[21] C. C. Paige and M. A. Saunders, *Solution of sparse indefinite systems of linear equations*, SIAM Journal on Numerical Analysis, 12 (1975), pp. 617–629.

[22] T. Schmelzer, *cvxcla: Critical line algorithm for portfolio optimisation*. https://github.com/cvxgrp/cvxcla, 2024. MIT License.

[23] ——, *The critical line algorithm: An operator-based implementation for the constrained mean-variance efficient frontier in cvxcla*. Working paper, 2026.

[24] ——, *Eigenvalue cleaning and direct solvers for long-only portfolio optimisation*. Companion paper, 2026.

[25] T. Schmelzer, M. Stoll, and M. Wolf, *Matrix-free methods for long-only portfolio optimization*. Companion paper, 2026.